Formative Assignment 1: Static MATLAB-Based FEM Modelling

Candidate Number: xxxxx
Date: xx/xx/xxxx

Contents

1 Local Element Matrix Functions **2**
1.1 Diffusion Operator 2
1.1.1 Diffusion Matrix Function Code: 2
1.1.2 Unit Testing 3
1.2 Linear Reaction Operator 4
1.2.1 Unit Test Code: 4
1.2.2 Linear Reaction Matrix Function Code: 6
1.2.3 Unit Testing 6

2 Solving Laplace's Equation using FEM **7**

3 Verifying FEM Solver for Diffusion-Reaction Equation **7**

1 Local Element Matrix Functions

1.1 Diffusion Operator

The equation for the local element matrix for the diffusion operator with linear basis functions, is as follows:

$$k_D \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \tag{1}$$

where:

$$k_D = \frac{2JD}{h^2} \qquad h = x_1 - x_0 \qquad J = \left| \frac{h}{2} \right| \tag{2}$$

1.1.1 Diffusion Matrix Function Code:

```
function [diff_mat] = LaplaceElemMatrix(D, eID, msh)
% function to create a local element matrix for the diffusion operator
% Inputs:
% D - diffusion coefficient
% eID - local element number
% msh - mesh data structure
% Output:
% diff_mat - local element matrix for the diffusion operator

% Calculate the spacial step size, h, from the x coordinate of each node
h = msh.elem(eID).x(2) - msh.elem(eID).x(1);

% Fetch the value of the Jacobian from the data structure
J = abs(msh.elem(eID).J);

% Calculate the magnitude of int00, int01, int10, int11
k_D = 2 * D / h^2 * J;

% Assemble the values into the matrix with appropriate signs
diff_mat = [k_D -k_D; -k_D k_D];
```

1.1.2 Unit Testing

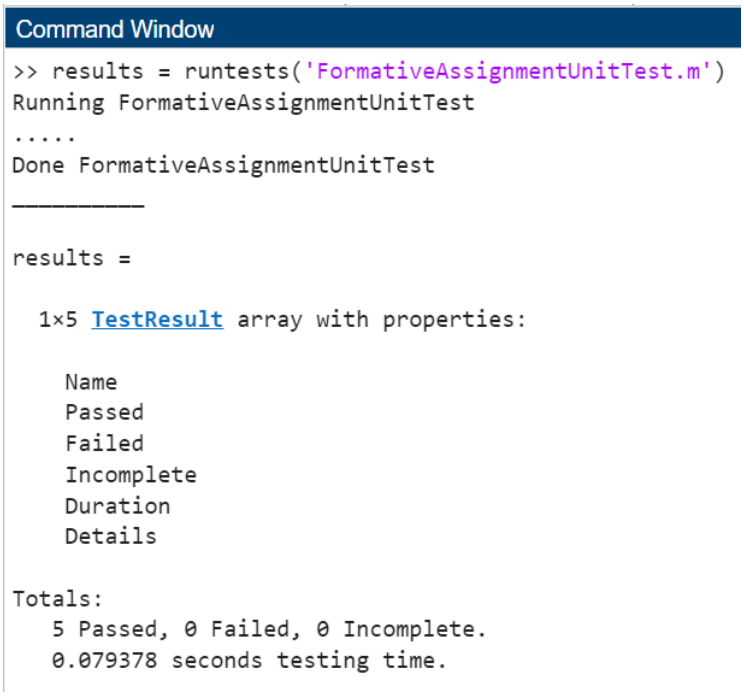

Figure 1: Result of running unit test with LaplaceElemMatrix.m

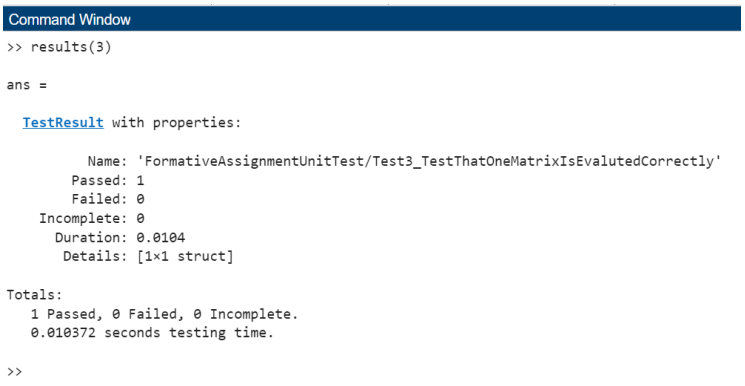

Figure 2: Example test result for Test 3 in unit test

Figures 1 and 2 shows that the local diffusion operator matrix function successfully passed all 5 of the unit tests provided for the task.

1.2 Linear Reaction Operator

The equation for the local element matrix for the linear reaction operator with linear basis functions is as follows:

$$k_\lambda \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \tag{3}$$

Where:

$$k_\lambda = \frac{J\lambda}{3} \qquad J = \left| \frac{x_1 - x_0}{2} \right| \tag{4}$$

The unit test function that was provided for the diffusion operator matrix may be replicated and modified for the linear reaction operator. Unit tests 1 and 2 remain valid for the linear reaction operator. Unit tests 3 -5 were updated with the following variables and expected results:

Unit Test	λ	J	Matrix
Test 3	9	0.333	$\begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$
Test 4	12	0.5	$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$
Test 5	12	0.0625	$\begin{bmatrix} 0.25 & 0.125 \\ 0.125 & 0.25 \end{bmatrix}$

1.2.1 Unit Test Code:

```
%% Test 1: test symmetry of the matrix
% % Test that this matrix is symmetric
tol = 1e-14;
lambda = 2; % linear reaction coefficient
eID=1; %element ID

msh = OneDimLinearMeshGen(0,1,10); % create equally spaced mesh
elemat = ReactionElemMatrix(lambda,eID,msh);  % form the local element matrix

assert(abs(elemat(1,2) - elemat(2,1)) <= tol)

%% Test 2: test 2 different elements of the same size produce same matrix
% % Test that for two elements of an equispaced mesh, as described in the
% % lectures, the element matrices calculated are the same
tol = 1e-14;
lambda = 5; % linear reaction coefficient
eID=1; %element ID

msh = OneDimLinearMeshGen(0,1,10);
elemat1 = ReactionElemMatrix(lambda,eID,msh);

eID=2; %element ID
elemat2 = ReactionElemMatrix(lambda,eID,msh);

% compare mesh at element ID 1 and 2
diff = elemat1 - elemat2;
diffnorm = sum(sum(diff.*diff));
assert(abs(diffnorm) <= tol)
```

```
%% Test 3: test that one matrix is evaluted correctly
% % Test that element 1 of the three element mesh problem described in the lectures
% % the element matrix is evaluated correctly
tol = 1e-14;
lambda = 9; % linear reaction coefficient
eID=1; %element ID

msh = OneDimLinearMeshGen(0,1,3);
elemat1 = ReactionElemMatrix(lambda,eID,msh);

elemat2 = [1 .5; .5 1]; % local matrix evaluated by hand
diff = elemat1 - elemat2; %calculate the difference between the two matrices
diffnorm = sum(sum(diff.*diff)); %calculates the total squared error between the matrices
assert(abs(diffnorm) <= tol)

%% Test 4: test that different sized elements in a mesh are evaluted correctly - element 1
% % Test that elements in a non-equally spaced mesh are evaluated correctly
tol = 1e-14;
lambda = 12; % linear reaction coefficient
eID=1; %element ID

msh = OneDimSimpleRefinedMeshGen(0,1,5); % Create a non-uniformly distributed mesh
elemat1 = ReactionElemMatrix(lambda,eID,msh);

elemat2 = [2 1; 1 2]; % Matrix for element 1, J = 0.25
diff = elemat1 - elemat2; %calculate the difference between the two matrices
diffnorm = sum(sum(diff.*diff)); %calculates the total squared error between the matrices
assert(abs(diffnorm) <= tol)

%% Test 5: test that different sized elements in a mesh are evaluted correctly - element 4
% % Test that elements in a non-equally spaced mesh are evaluated correctly
tol = 1e-14;
lambda = 12; % linear reaction coefficient
eID=4; %element ID

msh = OneDimSimpleRefinedMeshGen(0,1,5);
elemat1 = ReactionElemMatrix(lambda,eID,msh);

elemat2 = [0.25 0.125; 0.125 0.25]; % Matrix for element 4, J = 0.03125
diff = elemat1 - elemat2; %calculate the difference between the two matrices
diffnorm = sum(sum(diff.*diff)); %calculates the total squared error between the matrices
assert(abs(diffnorm) <= tol)
```

1.2.2 Linear Reaction Matrix Function Code:

```
function [reac_mat] = ReactionElemMatrix(lambda, eID, msh)
% function to create a local element matrix for the diffusion operator
% Inputs:
% lambda - reaction coefficient
% eID - local element number
% msh - mesh data structure
% Output:
% reac_mat - local element matrix for the diffusion operator

% Fetch the value of the Jacobian from the data structure
J = abs(msh.elem(eID).J);

% Calculate the magnitude k_lambda
k_lambda = J*lambda/3;

% Assemble the values into the matrix with appropriate signs
reac_mat = [2*k_lambda k_lambda; k_lambda 2*k_lambda];
```

1.2.3 Unit Testing

```
Command Window
   0.061045 seconds testing time.

>> result = runtests('LinearReactionUnitTest.m')
Running LinearReactionUnitTest
.....
Done LinearReactionUnitTest
__________

result =

  1×5 TestResult array with properties:

    Name
    Passed
    Failed
    Incomplete
    Duration
    Details

Totals:
   5 Passed, 0 Failed, 0 Incomplete.
   0.12657 seconds testing time.
```

Figure 3: Linear reaction operator unit testing result

Figure 3 shows that the function to create the local element matrix for the linear reaction operator passes all five of the unit tests and, therefore, matches the theory for varying spacial steps.

2 Solving Laplace's Equation using FEM

3 Verifying FEM Solver for Diffusion-Reaction Equation